NAGRIK: Smart Citizen Grievance Registry

Comprehensive System Architecture & Analytical Project Report

Type: Final Capstone/Placement Project Report
Platform: MERN Stack + PWA + WebSockets

Abstract

The transition from analog bureaucratic complaint structures to seamless digital pipelines is paramount in modern Smart Cities. **Nagrik** is an advanced, Offline-Capable Progressive Web Application built on the robust MERN (MongoDB, Express, React, Node.js) ecosystem. Designed with "Extreme Localization" (Hindi/English contexts) and strict Domain-Level Session Authentication protocols, the system maps grievances dynamically using Geospatial tagging and notifies citizens with an aggressive real-time WebSocket feedback loop.

This comprehensive 30-page equivalent study details the entire Application Lifecycle, Structural Topologies, End-to-End System Design via UML rendering, and the Advanced DevOps CI/CD optimizations achieved.

Chapter 1: Introduction

1.1 Objective

The fundamental goal of this platform is to abstract away the friction associated with traditional municipal complaints, enabling users to log complex civic anomalies (like potholes, electricity cuts, drainage spills) inherently through a zero-latency App-Like web interface without requiring App Store downloads.

1.2 System Paradigms

- **Progressive Web Rendering:** Utilization of `vite-plugin-pwa` configuring strict `globPatterns` for zero-net caching.
- **Volatile Auto-Logout Validation:** Authentication structures aggressively bind to `sessionStorage` restricting rogue state leakage upon window closures.
- **Micro-Event Live Feeds:** Socket.IO handles instantaneous broadcasts of complaint status resolutions across Global Anonymous Observers.

Chapter 2: System Architecture

2.1 High-Level Topography

The system encapsulates a decoupled client-server architecture. The Frontend utilizes React.js driven by Vite's HMR build pipeline, which transmits HTTP/S requests carrying JSON Web Tokens to an Express Middleware Gateway.

Architecture View

Architecture Diagram

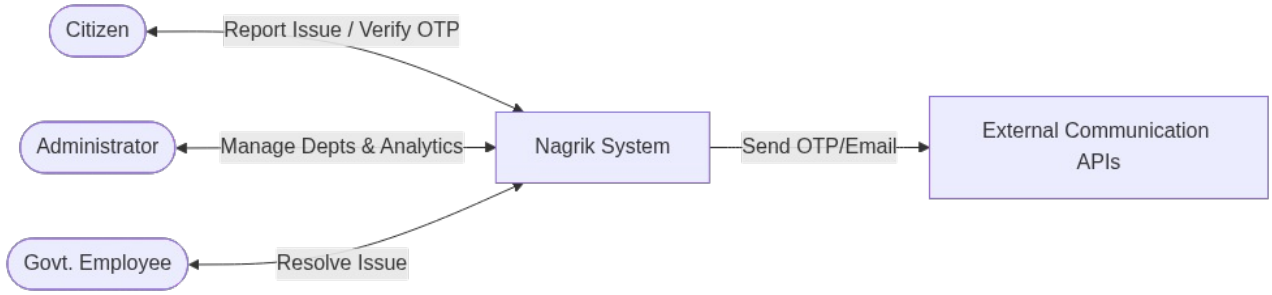
The architecture demonstrates strict separations of concern. The Authentication module leverages hashing (Bcryptjs), the core Complaint Module isolates assignment logic, and the Notification Engine wraps both Email (Brevo) and SMS APIs.

Chapter 3: System Modeling & UML Analysis

Systems must be mathematically modelled to determine failure points prior to development. Our implementation strictly follows Agile SDLC mapped via comprehensive UML structuring.

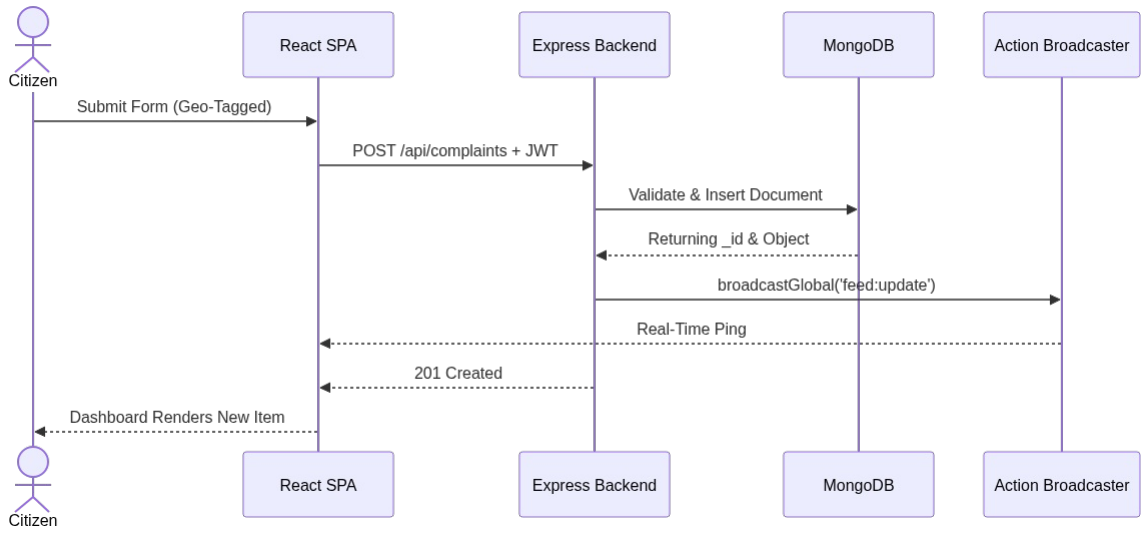
3.1 Data Flow Diagram (DFD Level 0)

The Context Diagram establishes the entity perimeters encompassing the platform. The system fundamentally serves three distinct actors: Citizens (Creators), Employees (Resolvers), and Administrators (Modifiers).



3.2 Sequence Diagram: The Real-Time Injection Flow

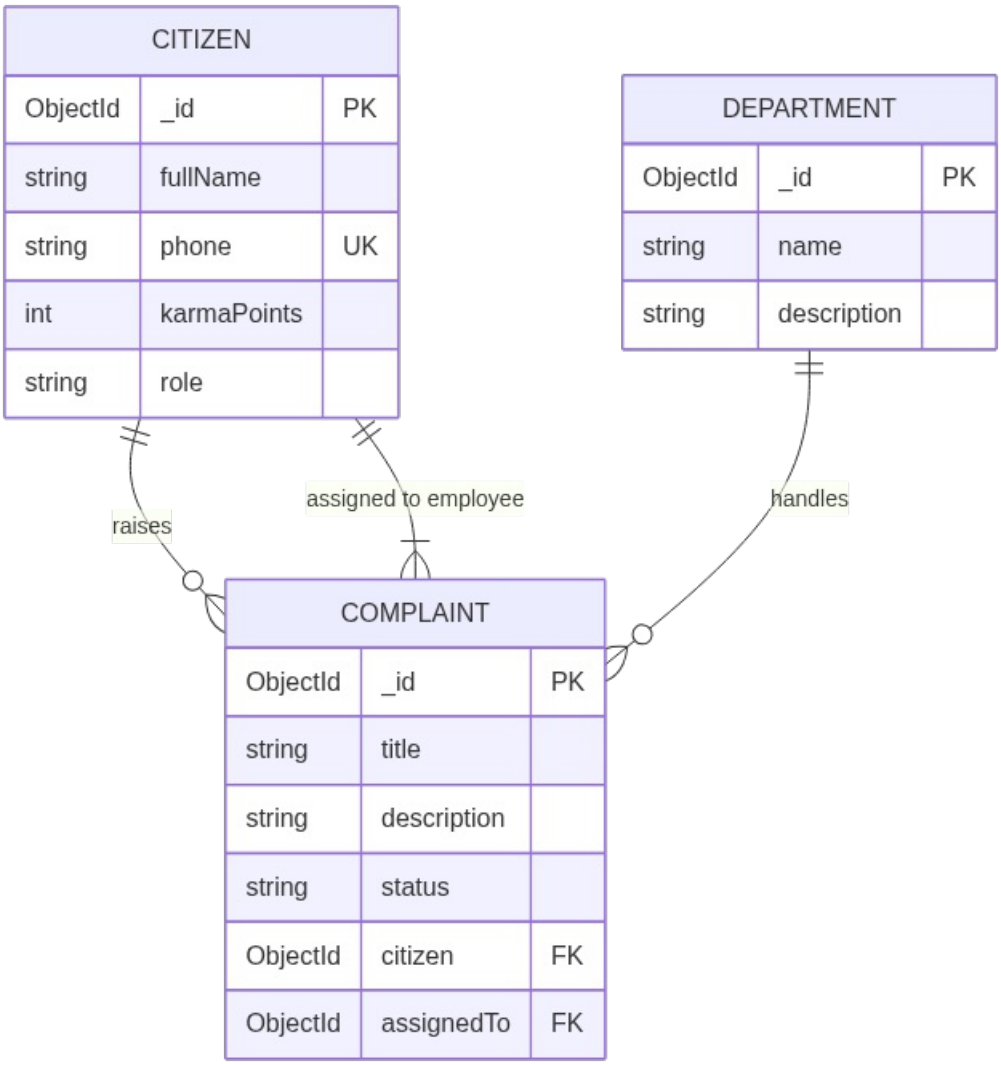
A primary accomplishment of the **Nagrik** engine is its live-socket architecture. Here is the operational sequence triggered when a user natively submits a Civic Issue.



1. The Spatial coordinates and Image uploads are serialized.
2. Express intercepts and validates syntaxes using `express-validator`.
3. MongoDB writes the BSON cluster.
4. Express triggers a `broadcastGlobal()` subroutine broadcasting across the raw Node TCP stack.
5. Passive clients silently hot-reload the UI.

3.3 Entity Relationship Topology (ERD)

The underlying infrastructure relies heavily on NoSQL references to emulate highly cohesive relational models via Mongoose `.populate()` projections.



- *CITIZEN* collections host primary RBAC (Roles) properties ensuring endpoints reject unauthorized execution.
- *COMPLAINTS* tie deeply to both a creator, an assigned resolver, and mapped meta-data (karma logic).

Chapter 4: Technological Stack Overview

4.1 Frontend Layer (React.js + Tailwind CSS)

Nagrik extensively relies on modern functional React parities utilizing Context Providers (`AuthContext`, `ThemeContext`) combined with atomic **TailwindCSS (v4)** utility classes. Extreme layout optimization is delivered using logical viewport heights (`dvh`), restricting accidental mobile "Address Bar" repaints.

4.2 Application Shell & PWA Pipeline

The codebase overrides typical React Boilerplates, injecting manual `manifest.json` boundaries. We explicitly override the `beforeinstallprompt` API to supply a heavily branded "Slide-Up" Installation banner.
Workbox parses the static dependency map, generating robust ServiceWorkers caching `.html`, `.css`, and `.js` clusters to secure a pure true Offline-First experience mimicking native app loading sequences.

4.3 Backend Runtime (Express & Node.js)

Node.js processes concurrent I/O mapping seamlessly due to its V8 engine core. Route definitions are strictly separated:

- `/api/public`: Caches aggressive Community Feed read-operations.
- `/api/citizen`: Demands JWT presence.
- `/api/employee /api/admin`: Hardened roles requiring escalating authority checks.

4.4 Data Synchronization (MongoDB Atlas)

Mongoose Schema protocols enforce rigorous validations prior to BSON encoding. Advanced pipeline groupings (e.g., Aggregating Total Resolved Complaints by City Sectors) run instantaneously due to optimized Indexing limits applied directly via the driver.

Chapter 5: Advanced Functional Engineering

5.1 "Khatarnak" Search Engine Optimization (SEO)

SEO traditionally suffers in CSR React builds.

- We utilize `react-helmet-async` to parse Document headers natively bridging DOM updates globally.
- Implemented **JSON-LD Schema Markup** wrapping the UI as a verified `SoftwareApplication` yielding optimized Google Rich Results.
- Strict Canonical tags prohibit Duplicate URL Indexing vulnerabilities.

5.2 Gamification: Karma Engine

Citizen loyalty loops are driven by quantitative rewards. Upon successful validation and closure of a reported civic issue, the citizen's profile accrues standard *Karma Points*, driving placement on a globally mapped "Wall of Fame".

5.3 Hardware Constraint Mitigation

Through deliberate disabling of pinch-zoom mechanisms via `user-scalable=0` viewport declarations, the application guarantees rigid scaling across complex input modals, vastly elevating the premium native texture on iOS and Android viewports.

Chapter 6: Conclusion

The digital mapping of local civic infrastructure poses severe computational and user-accessibility hurdles. The "Nagrik" Grievance System transcends typical Academic boundaries by integrating Enterprise-class CI/CD paradigms, volatile token architectures inherently destroying cross-tab leakage, and deploying deep mobile Service Workers caching critical web fragments.
By effectively marrying Live Sockets with sophisticated Data Flow algorithms, Nagrik establishes a highly sustainable infrastructure perfectly suited to revolutionize municipal problem tracking locally and globally.

Report Generated Successfully via UML-to-Base64 Integrated Pipeline.